



Análisis de Resultados: Algoritmos de Partición de Palabras

Curso 2025/2026

Práctica 2: Programación Dinámica

Algoritmia Básica

Autores:

Imad Habib Jali

Jorge Bellido Lobera

901421@unizar.es

903080@unizar.es

1. Introducción y Descripción de Variantes

En este informe se analiza la corrección y eficiencia computacional de tres implementaciones diseñadas para resolver el problema de la Partición de Palabras (Word Break). El objetivo es segmentar una cadena de caracteres sin espacios en una secuencia de palabras válidas pertenecientes a un diccionario.

Se han desarrollado las siguientes tres variantes:

- Variante 1 (Recursiva Pura - Fuerza Bruta): Explora sistemáticamente todos los cortes posibles. Su complejidad teórica es exponencial $O(2^n)$ en el peor de los casos, dado que incurre en la superposición de subproblemas (recalcula particiones idénticas múltiples veces).
- Variante 2 (Recursiva con Memoización - Top-Down): Mejora la Variante 1 introduciendo una tabla hash (`std::unordered_map`) para cachear los resultados de los sufijos ya procesados. Reduce la complejidad temporal a polinómica $O(n^3)$ a expensas de un incremento en la complejidad espacial.
- Variante 3 (Tabulación - Bottom-Up): Elimina la recursividad empleando un enfoque iterativo. Construye la solución desde las subcadenas más pequeñas hasta la longitud total de la cadena usando un vector de memorización, asegurando un uso más seguro y predecible de la memoria (evita el desbordamiento de la pila de llamadas).

2. Corrección del Algoritmo (Casos del Enunciado)

La corrección algorítmica se ha validado mediante un diccionario base y tres textos predefinidos. Las tres variantes devuelven exactamente los mismos resultados lógicos:

- Texto 1 ("megusta", len: 7): Encuentra la única partición válida ("me gusta").
- Texto 2 ("megustasoldar", len: 13): Es capaz de encontrar la ambigüedad semántica y devolver las dos ramas válidas ("me gusta sol dar" y "me gusta soldar").
- Texto 3 (len: 12): Detecta correctamente la imposibilidad de partición (0 soluciones).

Análisis de rendimiento en casos pequeños: Se observa que para longitudes reducidas ($N \leq 14$), los tiempos son casi instantáneos (1-5us). Paradójicamente, la Variante 1 es ligeramente más rápida (1-4us) que las variantes optimizadas (2-5us). Esto se debe a que el overhead (sobrecoste) computacional de reservar memoria dinámica e instanciar las estructuras de datos (mapas hash en V2 y vectores bidimensionales en V3) es mayor que el beneficio obtenido de no recalcular subproblemas en cadenas tan sumamente cortas.

3. Escalabilidad y Rendimiento (Batería Aleatoria)

Para comprobar el verdadero coste de los algoritmos, se han generado diccionarios de 100 y 1000 palabras, con frases de longitud proporcional (10 y 100 palabras concatenadas respectivamente).

Diccionarios Pequeños (100 palabras)

Las tres variantes muestran un rendimiento similar y muy bajo (14-23us). Al ser frases cortas, el árbol de decisión no llega a ramificarse lo suficiente como para que la explosión combinatoria de la Variante 1 sea perjudicial. Nuevamente, la inicialización de estructuras en V2 y V3 supone una ligera penalización de 4us frente a la fuerza bruta.

Diccionarios Grandes (1000 palabras / Textos largos)

En este escenario se evidencia la necesidad crítica de la Programación Dinámica.

- La Variante 1 ha sido omitida por el sistema de control (TIMEOUT), ya que la explosión combinatoria $O(2^n)$ para un texto de 100 palabras concatenadas requiere un tiempo de ejecución inviable (potencialmente años de CPU).
- La Variante 2 (Memoización) y la Variante 3 (Tabulación) resuelven el problema de forma óptima en 2000us (2 milisegundos). El tiempo crece polinómicamente frente al incremento lineal de los datos de entrada, confirmando la optimización teórica.

4. Análisis de Escenarios: Textos Limpios vs. Sucios

El entorno de pruebas incluyó mutaciones en los textos ("textos sucios") alterando caracteres con una probabilidad de $1/(LF \cdot 10)$, donde LF es la longitud de la frase.

- Comportamiento de la Variante 2 (Top-Down): Curiosamente, la Variante 2 tiende a ser más rápida en escenarios "sucios" (ej. 1812us) que en escenarios "limpios" (1973us). Esto ocurre porque, al encontrar un carácter mutado que no encaja en el diccionario, la recursividad "poda" (descarta) esa rama de exploración tempranamente y finaliza sin tener que reconstruir soluciones válidas complejas.
- Comportamiento de la Variante 3 (Bottom-Up): Por el contrario, la Variante 3 muestra tiempos ligeramente superiores en algunos casos sucios (2082us). Al ser un algoritmo tabular iterativo, la matriz bidimensional siempre recorre el bucle principal de 1 a N. Aunque los estados internos se descartan rápido si la casilla anterior está vacía, el bucle debe ejecutarse irremediablemente, haciéndola menos flexible ante cortes tempranos comparado con la recursión con memoización.

5. Conclusiones

1. Imprescindibilidad de la Programación Dinámica: La Variante 1 es útil únicamente como modelo mental y para casos triviales. Para sistemas reales, la superposición de subproblemas exige cacheo, haciendo de la Programación Dinámica una herramienta obligatoria.
2. Top-Down vs Bottom-Up: Ambas técnicas logran reducir la complejidad de exponencial a polinómica. La variante tabular (Bottom-Up) es preferible en arquitecturas con pila de llamadas limitadas, mientras que la memoización (Top-Down) puede mostrar ligeras ventajas cuando la cadena es inválida y permite podas drásticas del árbol de decisión.
3. Costes fijos: La optimización no es gratuita. Las estructuras de datos introducen un sobre coste que penaliza a los algoritmos eficientes frente a la fuerza bruta en instancias muy reducidas del problema.